

44.6 Generalized additive models

Everything so far assumed a normal response and an identity link. Lifting that costs one paragraph, because [Part VIII](#) did the work: a generalized linear model is fitted by repeated weighted least squares (Theorem 34.4.1), and repeated *penalized* weighted least squares is what this chapter has been doing throughout. Put the two together and the model class that carries the name appears.

44.6.1 The model and its fit

Theorem 44.6.1 — Generalized additive models

Let $y_1, \dots, y_n$ be independent from an exponential dispersion family in the sense of Definition 34.2.1, with dispersion ϕ , variance function V , link g and mean $\mu_i = g^{-1}(\eta_i)$, and let $\boldsymbol{\eta} = \mathbf{Z}\boldsymbol{\gamma}$ be the structured additive predictor Eq. 44.5.1 with block penalty $\mathbf{P}$ as in Eq. 44.1.5. Define the **penalized deviance**

$$Q(\boldsymbol{\gamma}) = D(\boldsymbol{\gamma}) + \boldsymbol{\gamma}^\top \mathbf{P}\boldsymbol{\gamma}, \quad (44.6.1)$$

with D the deviance of Definition 34.5.1. Assume that $\mathbf{Z}^\top \mathbf{W}\mathbf{Z} + \mathbf{P}$ is positive definite at every iterate and at the limit — the penalty supplies what the basis lacks, as in Proposition 44.1.1 — and let $\hat{\boldsymbol{\gamma}}$ be a minimizer of [Eq. 44.6.1](#), which then exists and is locally unique.

- (a) Minimizing [Eq. 44.6.1](#) is the same as maximizing the **penalized log-likelihood** $\ell(\boldsymbol{\gamma}) - (2\phi)^{-1} \boldsymbol{\gamma}^\top \mathbf{P}\boldsymbol{\gamma}$. Neither the minimizer nor the iteration below depends on ϕ .
- (b) (*Penalized IRLS.*) With $\mathbf{W}$ the working weights and $\mathbf{z}$ the working response of Eq. 34.4.1, both evaluated at $\boldsymbol{\gamma}^{(t)}$, the Fisher scoring step for [Eq. 44.6.1](#) is

$$\boldsymbol{\gamma}^{(t+1)} = (\mathbf{Z}^\top \mathbf{W}\mathbf{Z} + \mathbf{P})^{-1} \mathbf{Z}^\top \mathbf{W}\mathbf{z}, \quad (44.6.2)$$

that is, the penalized weighted least squares fit of the working response on the same design. At a fixed point the unpenalized columns satisfy $\mathbf{X}^\top \mathbf{W}\mathbf{D}^{-1}(\mathbf{y} - \boldsymbol{\mu}) = \mathbf{0}$, where $\mathbf{D} = \text{diag}\{h'(\eta_i)\}$; for the canonical link with prior weights $w_i \equiv 1$ this is $\mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu}) = \mathbf{0}$.

- (c) (*Degrees of freedom.*) At convergence, with $\mathbf{A} = \mathbf{Z}^\top \mathbf{W}\mathbf{Z} + \mathbf{P}$, the effective degrees of freedom are $\text{df} = \text{tr}(\mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W}\mathbf{Z})$ and split over terms exactly as in Proposition 44.1.2(b). The approximate covariance of $\hat{\boldsymbol{\gamma}}$ is $\phi \mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W}\mathbf{Z} \mathbf{A}^{-1}$ in its frequentist form and $\phi \mathbf{A}^{-1}$ in the Bayesian form of Proposition 44.5.2.
- (d) (*Inference for a term.*) Let $\mathbf{V}_j = \phi(\mathbf{A}^{-1})_{jj}$ and let $\mathbf{V}_j^{r-}$ be the rank- r pseudo-inverse of $\mathbf{V}_j$ with r the integer nearest to df_j . Under $f_j \equiv 0$, and treating $\boldsymbol{\lambda}$ as fixed and ϕ as known, $T_j = \hat{\boldsymbol{\gamma}}_j^\top \mathbf{V}_j^{r-} \hat{\boldsymbol{\gamma}}_j$ has approximately the $\chi^2(r)$ distribution; with ϕ estimated, T_j/r is referred to $F(r, n - \text{df})$. This is an approximation, stated here without proof; see the notes.

Proof.

- (a) By Definition 34.5.1, $D(\boldsymbol{\gamma}) = 2\phi\{\ell_{\text{sat}} - \ell(\boldsymbol{\gamma})\}$ with ℓ_{sat} free of $\boldsymbol{\gamma}$; dividing Eq. 44.6.1 by 2ϕ and dropping the constant gives the stated criterion, and rescaling an objective by a positive constant does not move its minimizer.
- (b) Let $\mathbf{U} = \partial\ell/\partial\boldsymbol{\gamma}$ and $\boldsymbol{\mathcal{I}} = \mathbf{Z}^\top \mathbf{W} \mathbf{Z} / \phi$ be the score and expected information of the unpenalized model (Theorem 34.3.1), so $\partial D / \partial \boldsymbol{\gamma} = -2\phi \mathbf{U}$ and $E(\partial^2 D / \partial \boldsymbol{\gamma} \partial \boldsymbol{\gamma}^\top) = 2\mathbf{Z}^\top \mathbf{W} \mathbf{Z}$. Fisher scoring for Eq. 44.6.1 therefore takes

$$\begin{aligned}\boldsymbol{\gamma}^{(t+1)} &= \boldsymbol{\gamma}^{(t)} - (2\mathbf{Z}^\top \mathbf{W} \mathbf{Z} + 2\mathbf{P})^{-1}(-2\phi \mathbf{U} + 2\mathbf{P}\boldsymbol{\gamma}^{(t)}) \\ &= \mathbf{A}^{-1}\{\mathbf{A}\boldsymbol{\gamma}^{(t)} + \phi \mathbf{U} - \mathbf{P}\boldsymbol{\gamma}^{(t)}\},\end{aligned}$$

and $\mathbf{A}\boldsymbol{\gamma}^{(t)} - \mathbf{P}\boldsymbol{\gamma}^{(t)} = \mathbf{Z}^\top \mathbf{W} \mathbf{Z} \boldsymbol{\gamma}^{(t)} = \mathbf{Z}^\top \mathbf{W} \boldsymbol{\eta}^{(t)}$, while $\phi \mathbf{U} = \mathbf{Z}^\top \mathbf{W} \mathbf{D}^{-1}(\mathbf{y} - \boldsymbol{\mu})$ by Theorem 34.3.1. Adding,

$$\boldsymbol{\gamma}^{(t+1)} = \mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W} \{\boldsymbol{\eta}^{(t)} + \mathbf{D}^{-1}(\mathbf{y} - \boldsymbol{\mu})\} = \mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W} \mathbf{z}^{(t)},$$

which is Eq. 44.6.2; ϕ has cancelled, proving the last claim of (a). At a fixed point the gradient of Eq. 44.6.1 vanishes, that is $\mathbf{Z}^\top \mathbf{W} \mathbf{D}^{-1}(\mathbf{y} - \boldsymbol{\mu}) = \mathbf{P}\boldsymbol{\gamma}$; the rows of $\mathbf{P}$ belonging to $\mathbf{X}$ are zero, so those components read $\mathbf{X}^\top \mathbf{W} \mathbf{D}^{-1}(\mathbf{y} - \boldsymbol{\mu}) = \mathbf{0}$. For the canonical link $\mathbf{W} \mathbf{D}^{-1} = \text{diag}\{w_i h'(\eta_i) / V(\mu_i)\}$ reduces to $\text{diag}(w_i)$, since $h'(\eta) = V(\mu)$ there (Theorem 34.3.1(d)), and with $w_i \equiv 1$ the identity becomes $\mathbf{X}^\top(\mathbf{y} - \boldsymbol{\mu}) = \mathbf{0}$.

- (c) At convergence the fitted linear predictor is $\mathbf{Z} \mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W} \mathbf{z}$, of the same form as in Proposition 44.1.2; its trace is $\text{tr}(\mathbf{Z} \mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W}) = \text{tr}(\mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W} \mathbf{Z})$ (Theorem 1.6.1), and the block decomposition of Proposition 44.1.2(b) goes through with $\mathbf{Z}^\top \mathbf{W} \mathbf{Z}$ in place of $\mathbf{Z}^\top \mathbf{Z}$. Treating $\mathbf{z}$ as a response with covariance $\phi \mathbf{W}^{-1}$, as in Theorem 34.4.1, gives $\text{Cov}(\hat{\boldsymbol{\gamma}}) \approx \phi \mathbf{A}^{-1} \mathbf{Z}^\top \mathbf{W} \mathbf{Z} \mathbf{A}^{-1}$; the Bayesian form is Proposition 44.5.2 with $\mathbf{R} = \phi \mathbf{W}^{-1}$.
- (d) Not proved here; see the notes.

■

Idea. Penalized IRLS is the whole algorithm: a generalized additive model is a generalized linear model whose weighted least squares step has acquired a penalty. Deviance, residuals, dispersion, offsets, the canonical-link orthogonality all survive, because the working problem is the same working problem.

What Hastie and Tibshirani (1990) called **local scoring** differs only in solving the inner penalized weighted least squares problem by backfitting (Theorem 44.2.1) rather than by one direct solve — essential when a $d \times d$ solve was expensive, a matter of taste now. Exercise (B1) works out the correspondence.

The inner loop needs one addition to the toolkit: the working weights and the working response. The first cell below repeats, unchanged, the term constructor of Section 44.2 and the solver of Section 44.5, so that this page runs on its own; the second is the new part.

```
import numpy as np
from scipy.interpolate import BSpline

def bspline_basis(x, n_basis=15, degree=3, lo=None, hi=None):
    """n_basis B-splines of the given degree on equally spaced knots covering the
    data."""
    lo = x.min() if lo is None else lo
```

```

hi = x.max() if hi is None else hi
inner = np.linspace(lo, hi, n_basis - degree + 1)
h = inner[1] - inner[0]
knots = np.r_[lo - h * np.arange(degree, 0, -1), inner, hi + h * np.arange(1,
degree + 1)]
return BSpline.design_matrix(np.clip(x, lo, hi), knots, degree, extrapolate=
False).toarray()

def difference_penalty(d, order=2):
    """K = D'D for the order-th difference matrix D: the P-spline penalty."""
    D = np.diff(np.eye(d), n=order, axis=0)
    return D.T @ D

def spline_term(x, n_basis=15, order=2, weight=None):
    """A P-spline term: its design, its penalty, and the map from new covariate
    values to
    design rows. The constraint  $\sum_i f(x_i) = 0$  is absorbed into the basis, and
    weight = u turns the term into the varying-coefficient term  $f(x) u$ ."""
    lo, hi = x.min(), x.max()
    B = bspline_basis(x, n_basis, lo=lo, hi=hi)
    Q, _ = np.linalg.qr(B.sum(axis=0)[: , None], mode="complete")
    U = Q[:, 1:] # a basis of {gamma : sum_i f(x_i)
    = 0}
    Z = B if weight is None else B * np.asarray(weight)[: , None]

    def design(xnew, wnew=None):
        Bn = bspline_basis(np.asarray(xnew, float), n_basis, lo=lo, hi=hi)
        return (Bn if wnew is None else Bn * np.asarray(wnew)[: , None]) @ U

    return Z @ U, U.T @ difference_penalty(n_basis, order) @ U, design

def assemble(blocks, X, lambdas):
    """Stack the parametric columns and the terms into one design and one block
    penalty."""
    Z = np.column_stack([X] + [Zj for Zj, _ in blocks])
    K = np.zeros((Z.shape[1], Z.shape[1]))
    start = X.shape[1]
    for (_, Kj), lam in zip(blocks, lambdas):
        d = Kj.shape[0]
        K[start:start + d, start:start + d] = lam * Kj
        start += d
    return Z, K

class Fit:
    """One penalized solve: coefficients, components, operators and degrees of
    freedom."""

    def __init__(self, y, blocks, X, lambdas, W=None, z=None, operators=True):
        Z, K = assemble(blocks, X, lambdas)
        w = np.ones(len(y)) if W is None else W
        Zw = Z * w[:, None]
        B = Z.T @ Zw # the weighted cross-
        product
        A = B + K
        Ainv = np.linalg.inv(A)
        self.Z, self.K, self.A, self.w = Z, K, A, w
        self.gamma = Ainv @ (Zw.T @ (y if z is None else z))
        self.fitted = Z @ self.gamma

```

```

        self.edf_total = np.trace(Ainv @ B)                                # tr(S) without
forming S
        self.parts, self.edf, self.comp = [], [], []
        start = X.shape[1]
        for Zj, _ in blocks:
            sl = slice(start, start + Zj.shape[1])
            self.parts.append(Zj @ self.gamma[sl])
            self.edf.append(np.trace(Ainv[sl] @ B[:, sl]))
            start += Zj.shape[1]
        if operators:                                                    # the n x n smoothers
            C = Ainv @ Zw.T
            self.S = Z @ C
            start = X.shape[1]
            for Zj, _ in blocks:
                self.comp.append(Zj @ C[start:start + Zj.shape[1]])
                start += Zj.shape[1]
            self.beta = self.gamma[:X.shape[1]]

def null_dim(K, tol=1e-9):
    """The dimension of the penalty null space: the part of a term that is never
penalized."""
    e = np.linalg.eigvalsh(K)
    return int(np.sum(e < tol * max(e.max(), 1.0)))

```

[Run this code online](#)

```

def penalized_irls(y, blocks, X, lambdas, family="poisson", offset=0.0, iters=60,
tol=1e-10):
    """Penalized IRLS: at each step a penalized weighted least squares fit of the
working
response on the same design -- Chapter 34's IRLS with the penalty added."""
    eta = np.log(np.maximum(y, 0.5)) if family == "poisson" else np.zeros(len(y))
    for _ in range(iters):
        if family == "poisson":
            mu = np.exp(eta + offset)
            w = dmu = mu                                                # w = (dmu/deta)^2 / V(mu) = mu
        else:                                                            # binomial with the logit link
            mu = 1 / (1 + np.exp(-(eta + offset)))
            w = dmu = mu * (1 - mu)
        w = np.maximum(w, 1e-10)
        z = eta + (y - mu) / dmu                                        # the working response
        fit = Fit(z, blocks, X, lambdas, W=w, z=z, operators=False)
        done = np.max(np.abs(fit.fitted - eta)) < tol
        eta = fit.fitted
        if done:
            break
    fit.eta, fit.mu = eta, mu
    return fit

def pearson_dispersion(y, fit, family="poisson"):
    """The Pearson estimate of the dispersion, with the effective degrees of freedom
    """
    v = fit.mu if family == "poisson" else fit.mu * (1 - fit.mu)
    return np.sum((y - fit.mu) ** 2 / v) / (len(y) - fit.edf_total)

def poisson_deviance(y, mu):
    """The Poisson deviance of Chapter 34, with the 0 log 0 convention."""
    t = np.where(y > 0, y * np.log(np.maximum(y, 1e-300) / mu), 0.0)

```

```

return 2 * np.sum(t - (y - mu))

def irls_reml(y, blocks, X, family="poisson", offset=0.0, outer=40, tol=1e-7,
             dispersion=False):
    """Penalized IRLS with the smoothing parameters updated at every step: each
    working
    problem gets the update of reml_lambdas, with the dispersion of Chapter 38 in
    place of
    the nominal phi = 1 when dispersion is True."""
    lam = np.ones(len(blocks))
    for _ in range(outer):
        fit = penalized_irls(y, blocks, X, lam, family=family, offset=offset)
        phi = pearson_dispersion(y, fit, family) if dispersion else 1.0
        new = lam.copy()
        start = X.shape[1]
        for j, (_, Kj) in enumerate(blocks):
            d = Kj.shape[0]
            g = fit.gamma[start:start + d]
            tau2 = max(g @ Kj @ g, 1e-12) / max(fit.edf[j] - null_dim(Kj), 1e-6)
            new[j] = min(max(phi / tau2, 1e-8), 1e10)
            start += d
        done = np.max(np.abs(np.log(new) - np.log(lam))) < tol
        lam = new
        if done:
            break
    fit = penalized_irls(y, blocks, X, lam, family=family, offset=offset)
    fit.phi = pearson_dispersion(y, fit, family)
    return lam, fit

```

[Run this code online](#)

44.6.2 Choosing the smoothing parameters, and the dispersion

Eq. 44.5.4 was derived for a normal response. The standard device, the *performance iteration* of Gu (1992), applies it to the working problem: at each penalized IRLS step treat $\mathbf{z}$ as a normal response with covariance $\phi \mathbf{W}^{-1}$, update $\boldsymbol{\lambda}$ by Eq. 44.5.4 with the weights in place, and continue. That is what `irls_reml` does above. Recomputing $\boldsymbol{\lambda}$ by maximizing a properly defined restricted likelihood for the generalized model at every step is more stable and is what modern software does; Wood (2011) gives it.

One choice there matters more than it looks. For a Poisson or binomial response the nominal dispersion is $\phi = 1$, and taking that at face value when the data are overdispersed makes $\lambda_j = \hat{\phi} / \hat{\tau}_j^2$ too small by the factor $\hat{\phi}$, so every curve is undersmoothed. Chapter 38 built the repair: estimate ϕ by the Pearson statistic over the residual degrees of freedom (Proposition 38.3.2) and use *that* in the update, which is the quasi-likelihood position of Definition 38.2.1.

Example. (Physician visits, with two smooth terms)

Example (Physician visits in a health insurance experiment) fitted a Poisson log-linear model to the RAND Health Insurance Experiment, $n = 20190$ person-years, with every regressor entering linearly — among them a general disease index, which has no reason

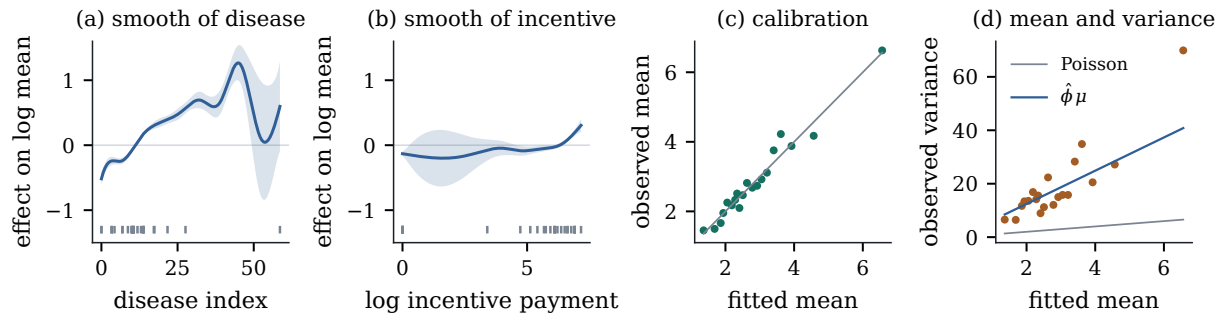


Figure 44.6.1: A Poisson generalized additive model for physician visits. (a) and (b) the two smooth terms, with two-standard-error bands and a rug of covariate quantiles. (c) and (d) observed mean, and observed variance, against fitted mean over twenty bins, the last with the Poisson and quasi-Poisson lines.

to be linear on the log scale. Take that index and one further variable of the same kind, the log of the participation incentive payment, and give each a P-spline term of 12 basis functions, keeping the coinsurance, deductible-plan, physical-limitation and health-status regressors linear: that is a generalized additive model, fitted by Eq. 44.6.2. Its baseline throughout is the **linear fit**, meaning the same list of regressors with the disease index and the incentive payment entering linearly too.

The disease term takes 9.11 effective degrees of freedom and the incentive term 4.48, out of 20.59 for the whole model. The estimated dispersion is 6.22, so these data are as overdispersed as Chapter 37 found them; the smoothing parameters, 1.16 and 103.94, were chosen with that dispersion in place. The coefficient of log coinsurance is -0.1146 , a rate ratio of 0.8917, against -0.0874 in the linear fit and -0.0712 in the smaller model of Example (Physician visits in a health insurance experiment), which carried no incentive-payment term at all; self-reported physical limitation still multiplies the rate by $e^{0.2777}$.

The deviance falls from 84378 for the linear fit to 83619, at a cost of 11.59 degrees of freedom. Scaling by the dispersion gives a quasi- F statistic of 10.54 on 11.59 and 20169 degrees of freedom, whose nominal p -value is 1.6×10^{-20} . The deviance difference is the statistic of Theorem 34.5.5(a), but its reference distribution there assumed a fixed nested pair, and here the larger model was smoothed. Read as a rough diagnostic rather than as a test, it says what panel (a) of Figure 44.6.1 shows: the effect of the disease index is far from linear — it rises steeply over the first half of the range and then flattens — and no straight line would do.

The same panel is a warning. Above a disease index of 40 lie 55 person-years, 0.3% of the sample, and the curve there swings by more than a unit on the log scale with a standard error of 0.347, against 0.011 at the median. That wiggle is an artefact of a handful of observations and a globally chosen smoothing parameter; report it as “no information above 40”, not as a finding.

```
import numpy as np
import statsmodels.api as sm

data = sm.datasets.randhie.load_pandas().data          # 20190 person-years, public
domain                                     # outpatient physician visits
y = data["mdvis"].to_numpy(float)                # outpatient physician visits
in the year
```

```

disea = data["disea"].to_numpy(float)           # a general disease index
lpi = data["lpi"].to_numpy(float)              # log participation incentive
      payment
linear = ["lncoins", "idp", "physlm", "hlthg", "hlthf", "hlthp"]
X = np.column_stack([np.ones(len(y))] + [data[v].to_numpy(float) for v in linear])

Z1, K1, design1 = spline_term(disea, n_basis=12)      # the two smooth terms
Z2, K2, design2 = spline_term(lpi, n_basis=12)
lambdas, fit = irls_reml(y, [(Z1, K1), (Z2, K2)], X, family="poisson", dispersion=
    True)

print("smoothing parameters", lambdas.round(2))
print(f"edf: disease index {fit.edf[0]:.2f}, incentive payment {fit.edf[1]:.2f}, "
      f"model {fit.edf_total:.2f}")
print(f"dispersion {fit.phi:.2f}")
for name, b in zip(linear, fit.beta[1:]):
    print(f" {name:8s} {b:8.4f}   rate ratio {np.exp(b):.4f}")

```

[Run this code online](#)

```

X_linear = np.column_stack([X, disea, lpi])      # the same regressors, both
      smooths linear
fit_linear = penalized_irls(y, [], X_linear, [], family="poisson")
D_gam = poisson_deviance(y, fit.mu)
D_linear = poisson_deviance(y, fit_linear.mu)
ddf = fit.edf_total - fit_linear.edf_total
F = (D_linear - D_gam) / ddf / fit.phi           # a quasi-F, not an exact
      null statistic
print(f"deviance {D_linear:.0f} (linear) against {D_gam:.0f} (smooth)")
print(f"quasi-F {F:.2f} on {ddf:.2f} and {len(y) - fit.edf_total:.0f} degrees of
      freedom")

```

[Run this code online](#)

44.6.3 What the p-values are worth

Warning (Inference after choosing the smoothing parameters). The statistic in [Theorem 44.6.1\(d\)](#) and the quasi- F of [Example \(Physician visits, with two smooth terms\)](#) both treat λ as a constant. It is not: it was chosen from the same data, by minimizing a criterion that rewards fitting those data well — Proposition 29.5.1 in a new costume. Three consequences, in order of importance:

1. The reference distribution is approximate even when everything else is right. How good the approximation of [Theorem 44.6.1\(d\)](#) is depends on the term; Wood (2013) reports both the cases where it works and the cases where it fails, and simulation is the only way to know which one is at hand.
2. A term that the smoothing-parameter selection has pushed to the boundary $\lambda_j = \infty$ — smoothed to a straight line, or to zero — sits on the edge of the parameter space, where Theorem 32.5.2 says the usual asymptotics fail.
3. A p -value for a term that was *selected* into the model by the same fit is not a p -value at all.

Use these numbers to rank terms and to screen, nothing more. For a claim that matters,

hold out data, or put the whole fitting procedure — smoothing-parameter selection included — inside the resampling loop of [Chapter 23](#).

44.6.4 Checking the fit

The model is still a generalized linear model at heart, so the diagnostics of [Chapter 39](#) transfer unchanged; panels (c) and (d) of [Figure 44.6.1](#) show two of them, binned by fitted mean. Panel (c) is the binned analogue of the exact recalibration identity [Proposition 35.4.2](#), which a penalized fit no longer satisfies exactly, since the score identity of [Theorem 44.6.1\(b\)](#) holds only for the unpenalized columns. Panel (d) puts the observed variance in the same bins against the fitted mean: far above the Poisson line $\sigma^2 = \mu$ and close to $\hat{\phi}\mu$, so the mean model is adequate, the variance model is not, and every standard error must carry the dispersion ([Definition 38.2.1](#)). One check is specific to these models: a term whose effective degrees of freedom sit close to $d_j - 1$, the most its basis allows, is asking for more basis functions; refit with more and see whether the curve moves.

A smoothing parameter shrinks a term towards its penalty null space but not through it: however large λ_j grows, a P-spline term with a second-difference penalty keeps a straight line and never leaves the model. Both standard repairs are penalties, so both live inside [Definition 44.5.1](#): a second penalty on the null space, letting the restricted likelihood shrink the term to zero ([Marra and Wood, 2011](#); [Exercise \(B2\)](#)); or a penalty that is not differentiable at zero, the group lasso of [Section 30.3](#) with each term a group, which removes terms exactly.

44.6.5 Where this leaves the book

Part IX set out to drop the linear predictor, and it has. A modern additive fit is a linear model in a basis, with a quadratic penalty per term, a variance ratio per penalty and a mixed model underneath; the response may come from any exponential dispersion family, and the terms may be curves, surfaces, maps or clusters.

One assumption has survived it all. Every model in this book, from [Chapter 5](#) to this page, has modelled the *mean* of the response and treated the rest of its distribution as a nuisance — σ^2 , or ϕ , or a variance function fixed by the family. When the spread or the shape is itself the object of interest, that is not enough. [Chapter 45](#), the last of the book, gives up the mean: it estimates quantiles directly, and then lets *every* parameter of a response distribution carry a structured additive predictor of its own ([Definition 45.5.1](#)) — this chapter’s machinery, run once per parameter.

44.6.6 Exercises

44.6.6.1 A. Check your understanding

Exercise A1.

In [Theorem 44.6.1\(a\)](#) the dispersion cancels from the iteration but not from the smoothing-parameter update. Explain the difference, and say what happens to the fitted curves if $\phi = 1$ is assumed when in truth $\phi = 6$.

Solution. Given λ , ϕ only rescales Eq. 44.6.1, so the minimizer is unchanged. The update $\lambda_j = \phi/\tau_j^2$ compares the penalty term with the noise level, so getting the noise level wrong by a factor of six makes every λ_j six times too small: undersmoothed curves, too many effective degrees of freedom, bands too narrow.

44.6.6.2 B. Practice

Exercise B1.

Local scoring. Show that replacing the direct solve in Eq. 44.6.2 by one or more backfitting cycles on the weighted penalized normal equations gives the local scoring algorithm of Hastie and Tibshirani, and that Theorem 44.2.1 applies to the inner loop with $\mathbf{Z}_j^\top \mathbf{Z}_j$ replaced by $\mathbf{Z}_j^\top \mathbf{W} \mathbf{Z}_j$. What must be true of $\mathbf{W}$ for the convergence proof to go through?

Solution. Eq. 44.6.2 is $\mathbf{A}\boldsymbol{\gamma} = \mathbf{Z}^\top \mathbf{W} \mathbf{z}$ with $\mathbf{A} = \mathbf{Z}^\top \mathbf{W} \mathbf{Z} + \mathbf{P}$: the structure of Eq. 44.1.6 with the inner product $\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{W}} = \mathbf{u}^\top \mathbf{W} \mathbf{v}$ in place of the Euclidean one, so backfitting on it is block Gauss–Seidel, which is local scoring. The proof of Theorem 44.2.1 needs $\mathbf{A}$ symmetric nonnegative definite with positive definite diagonal blocks, hence $\mathbf{W}$ nonnegative definite — working weights are, being $h'(\eta_i)^2/V(\mu_i) \geq 0$ — and positive on enough observations to keep each $\mathbf{Z}_j^\top \mathbf{W} \mathbf{Z}_j + \lambda_j \mathbf{P}_j$ nonsingular.

Exercise B2.

Shrinking a whole term away. Let $\mathbf{P}_j$ have null space of dimension m_j with orthonormal basis $\mathbf{Q}_j^0$, and replace the penalty by $\mathbf{P}_j + \epsilon \mathbf{Q}_j^0 \mathbf{Q}_j^{0\top}$ for a small $\epsilon > 0$. Show that the new penalty is positive definite, that $\lambda_j \rightarrow \infty$ now shrinks the whole term to zero, and that $\text{df}_j \rightarrow 0$.

Solution. Split $\boldsymbol{\gamma} = \boldsymbol{\gamma}^+ + \boldsymbol{\gamma}^0$ over the range and null space of $\mathbf{P}_j$; the new quadratic form is $\boldsymbol{\gamma}^{+\top} \mathbf{P}_j \boldsymbol{\gamma}^+ + \epsilon \|\boldsymbol{\gamma}^0\|^2$, zero only if both pieces vanish, so it is positive definite with null space $\{\mathbf{0}\}$. By Proposition 44.1.2(b) the term's degrees of freedom then fall to 0 as $\lambda_j \rightarrow \infty$, with $\hat{\boldsymbol{\gamma}}_j \rightarrow \mathbf{0}$: the restricted likelihood can select the term out, which is what Eq. 44.5.4 does when it explains nothing.

44.6.6.3 C. Going deeper

Exercise C1.

In Example (Physician visits, with two smooth terms) the curve above a disease index of 40 rests on 55 observations. Suppose all of them were deleted. Using the fact that the P-spline penalty of Definition 43.3.1 has the straight lines as its null space, describe exactly what the fitted curve does over the empty range, and compute the limit of its variance as the range of emptiness grows.